

markpublish Benutzerhandbuch

Moderne PDF- und HTML-Dokumentenerstellung aus Markdown

Dieses Handbuch bietet eine praxisorientierte Anleitung und vollständige Referenz zur Konfiguration, Template-Anpassung und Dokumentengenerierung mit markpublish.

AUTOR
Frank Winter

STATUS
Freigegeben

VERSION
1.0.0

DATUM
01.09.2026

COPYRIGHT
© 2026 Frank Winter

Inhaltsverzeichnis

1 Einführung & Architektur	5
1.1 Motivation & Zielsetzung	5
1.1.1 Kernvorteile auf einen Blick:	5
1.2 Die Verarbeitungs-Pipeline	5
2 Installation & Schnellstart	7
2.1 Voraussetzungen	7
2.2 Installation via pip	7
2.3 Erstes Projekt initialisieren	7
2.4 Dokument erstellen	8
3 Die CLI-Befehle	10
3.1 Befehlsübersicht	10
3.2 markpublish init	10
3.3 markpublish cheatsheet	10
3.4 markpublish build	11
3.4.1 Argumente & Optionen:	11
3.4.2 Beispiele:	11
3.5 markpublish templates	11
3.6 markpublish export-template	12
3.7 markpublish labels	12
3.7.1 Argumente & Optionen	12
4 Konfigurations-Leitfaden	14
4.1 Das document-Objekt	14
4.1.1 Automatische Datumsauflösung	14
4.1.2 Zusätzliche Metadatenfelder	14
4.2 Kapitel, Unterkapitel und Parts	15
4.2.1 Kapitel-Definition	15
4.2.2 Hierarchische Unterkapitel	15
4.2.3 Übergeordnete Abschnitte (Parts / Blöcke)	15
5 Das Template- und Design-System	18
5.1 Theme-basierte Verzeichnisstruktur	18
5.2 Die 3-stufige Auflösungs-Hierarchie	18
5.2.1 Konfiguration des gemeinsamen Template-Ordners	19
5.3 Statische Texte und Sprache	19
5.3.1 Die 3-stufige i18n-Kaskade	19
5.3.2 Das gemeinsame Format	20
5.3.3 Beispiel: PDF und HTML unterschiedlich beschriften	21

5.3.4 Freie Labels: eigene Texte des Templates	21
5.3.5 Die aufgelöste Tabelle ansehen	22
5.3.6 Verfügbare Schlüssel	23
5.4 Kopf- und Fußzeilen	24
5.4.1 Geometrie	25
5.4.2 Standardbelegung des Themes	26
6 Markdown-Features & Syntax	28
6.1 Code-Blöcke & Syntax Highlighting	28
6.2 Tabellen	28
6.3 GitHub-Style Callout-Boxen (Alerts)	28
6.4 Tasklisten	29
Anhänge	
6.5 Anhang A: YAML-Schema-Referenz	31
6.5.1 Dokumenten-Eigenschaften (document)	31
6.5.2 Kapitel- und Part-Eigenschaften (chapters)	32
6.5.2.1 break_before — wie ein Kapitel abgesetzt wird	32
6.5.2.2 toc_depth — Anhänge im Inhaltsverzeichnis flach halten	33
6.6 Anhang B: Fehlerbehebung & FAQ	34
6.6.1 WeasyPrint & GTK-Laufzeitumgebung auf Windows	34
6.6.1.1 Problem: cannot load library 'libgobject-2.0-0'	34
6.6.2 Bilder werden im PDF nicht angezeigt	34
6.6.2.1 Problem: Leere Bildrahmen oder fehlende Grafiken	34
6.6.3 Inhaltsverzeichnis zeigt falsche Seitenzahlen	34

KAPITEL 1

Einführung & Architektur

Überblick über Zielsetzung, Kernkonzepte und die modulare Verarbeitungs-Pipeline.

1 Einführung & Architektur

Willkommen beim **markpublish Benutzerhandbuch**. `markpublish` ist ein modernes, quelloffenes Publishing-Werkzeug, das aus einfachen Markdown-Dateien professionell gesetzte **PDF-Publikationen** und **HTML-Vorschauen** erzeugt.

1.1 Motivation & Zielsetzung

Viele Dokumentationswerkzeuge erfordern entweder komplexe LaTeX-Setups oder beschränken sich auf reine HTML-Webseiten. `markpublish` verbindet die Einfachheit von Markdown mit der typografischen Präzision von **CSS Paged Media** über die [WeasyPrint](#)-Engine.

1.1.1 Kernvorteile auf einen Blick:

- **Modulare Kapitel:** Jedes Kapitel wird als separate Markdown-Datei gepflegt.
- **Hierarchische Struktur:** Beliebig tief schachtelbare Unterkapitel und übergeordnete Abschnitte (*Parts*).
- **Deklarative Steuerung:** Ein zentrales Manifest (`markpublish.yaml`) steuert Inhalt, Metadaten und Layout.
- **W3C CSS Paged Media:** Exakte Kontrolle über `@page`-Ränder, mehrzeilige Kopf- und Fußzeilen, Seitenzahlen und Trennseiten.
- **Erweiterbare Pipeline:** Saubere Trennung zwischen Parser, Renderer und Template-Ebenen.

1.2 Die Verarbeitungs-Pipeline

Die Architektur von `markpublish` folgt einem mehrstufigen Prozess:

1. **Konfigurations-Lader (`config.loader`):** Liest die YAML-Struktur ein, validiert Datentypen und löst dynamische Variablen (z. B. `date: auto`) auf.
2. **Template-Resolver (`templates.resolver`):** Sucht nach der 3-stufigen Priorität (*User > Common/Workspace > Package*) nach dem passenden Theme für das Zielformat.
3. **Markdown- & TOC-Engine (`markdown.engine`):** Parst Markdown mit erweiterten Erweiterungen (Callouts, Tabellen, Pygments-Syntax-Highlighting), vergibt konsistente Überschriftennummern (`1.1` , `1.2`) und baut Inhaltsverzeichnisse auf.
4. **Renderer (`renderers.pdf` & `renderers.html`):** Übergibt die gerenderten HTML- und CSS-Fragmente an WeasyPrint für den PDF-Export oder speichert eigenständiges, responsives HTML.

KAPITEL 2

Installation & Schnellstart

Schritt-für-Schritt-Anleitung zur Installation und Erstellung des ersten Dokuments.

AUF EINEN BLICK

2.1 Voraussetzungen	7
2.2 Installation via pip	7
2.3 Erstes Projekt initialisieren	7
2.4 Dokument erstellen	8

2 Installation & Schnellstart

In diesem Kapitel erfahren Sie, wie Sie `markpublish` installieren und in weniger als zwei Minuten Ihr erstes Dokument kompilieren.

2.1 Voraussetzungen

`markpublish` benötigt **Python 3.10 oder neuer**. Es läuft nativ unter: - **Windows** (10, 11, Server) - **macOS** (Intel und Apple Silicon) - **Linux** (Debian, Ubuntu, Fedora, Arch, etc.)

2.2 Installation via pip

Installieren Sie das Paket über den Python-Paketmanager:

```
pip install markpublish
```

Für Entwickler oder zur Installation aus dem Quellcode:

```
git clone https://github.com/your-username/markpublish.git
cd markpublish
pip install -e ".*[dev]"
```

2.3 Erstes Projekt initialisieren

Verwenden Sie den Befehl `init`, um eine neue Dokumentenstruktur zu erzeugen:

```
markpublish init mein-leitfaden --title "Mein Leitfaden"
cd mein-leitfaden
```

Dieser Befehl erstellt die folgende Verzeichnisstruktur:

```
mein-leitfaden/
├─ markpublish.yaml          # Das Dokumenten-Manifest
├─ chapters/                # Markdown-Kapiteldateien
│   ├─ 01_introduction.md
│   ├─ 02_architecture.md
│   │   ├─ 02_1_details.md
│   └─ 03_appendix.md
```

2.4 Dokument erstellen

Rendern Sie Ihr Dokument mit dem `build`-Befehl:

```
# PDF erstellen (Standard)
markpublish build

# HTML-Vorschau erzeugen
markpublish build --target html

# Sowohl PDF als auch HTML in einem Durchgang bauen
markpublish build --target all
```

Die fertigen Dateien werden direkt im Projektordner abgelegt.

KAPITEL 3

Die CLI-Befehle

Detaillierte Erläuterung aller Befehle: build, init, templates und export-template.

AUF EINEN BLICK

3.1 Befehlsübersicht	10
3.2 markpublish init	10
3.3 markpublish cheatsheet	10
3.4 markpublish build	11
3.5 markpublish templates	11
3.6 markpublish export-template	12
3.7 markpublish labels	12

3 Die CLI-Befehle

`markpublish` bietet eine schlanke, intuitive Befehlszeilenschnittstelle (CLI), die über `markpublish` oder die Kurzform `mpub` aufgerufen werden kann.

3.1 Befehlsübersicht

BEFEHL	KURZBESCHREIBUNG
<code>markpublish init</code>	Erzeugt einen minimalen Projektstumpf (zwei Dateien)
<code>markpublish cheatsheet</code>	Rendert die zweiseitige Kurzreferenz
<code>markpublish build</code>	Kompiliert das Dokument zu PDF und/oder HTML
<code>markpublish templates</code>	Listet alle gefundenen Templates und deren Quellen auf
<code>markpublish export-template</code>	Exportiert ein Template zur individuellen Anpassung
<code>markpublish labels</code>	Zeigt die aufgelösten statischen Texte und ihre Herkunft

3.2 `markpublish init`

Legt einen minimalen Projektstumpf an: eine `markpublish.yaml` und ein Kapitel `next-steps.md`, beide direkt im Zielordner. Kein `chapters/`-Verzeichnis, keine Beispielkapitel.

```
markpublish init [ZIELORDNER] [--title "Titel"]
```

Der Stumpf ist absichtlich klein, weil er dazu da ist, gelöscht zu werden – spätestens beim ersten echten Kapitel. Die Referenz liegt deshalb nicht im Stumpf, sondern in `markpublish cheatsheet`; sowohl die YAML als auch das Kapitel verweisen darauf. Cover und Inhaltsverzeichnis sind abgeschaltet, weil ein einseitiger Entwurf beides nicht braucht.

3.3 `markpublish cheatsheet`

Rendert die mitgelieferte Kurzreferenz: Seite 1 die Schlüsseler der `markpublish.yaml`, Seite 2 die Grundlagen zu Themes und Templates.

```
markpublish cheatsheet [--target pdf|html|all] [--output PFAD] [--theme NAME]
```

Die Quelle liegt im Paket und wird bei jedem Aufruf neu gerendert. Damit passt die Referenz immer zur installierten Version, und ein erfolgreicher Lauf ist zugleich der Nachweis, dass die Rendering-Kette funktioniert – unter Windows also, dass WeasyPrint seine GTK-Laufzeit findet. Geschrieben wird nur das fertige Dokument in das aktuelle Verzeichnis; Quelldateien landen nie im Projekt des Nutzers.

Mit `--theme` rendert die Referenz in einem eigenen Theme statt im mitgelieferten. Fällt der PDF-Weg mangels GTK aus, liefert `--target html` dieselbe Referenz ohne WeasyPrint.

3.4 markpublish build

Kompiliert eine `markpublish.yaml`-Konfiguration.

```
markpublish build [CONFIG_FILE] [OPTIONEN]
```

3.4.1 Argumente & Optionen:

- `CONFIG_FILE` (*optional*, Standard: `markpublish.yaml`): Pfad zur YAML-Datei.
- `--target / -t` (Standard: `pdf`): Zielformat (`pdf`, `html` oder `all`).
- `--output / -o`: Benutzerdefinierter Ausgabepfad (Datei oder Verzeichnis).
- `--templates-dir`: Spezifischer Pfad zu einem gemeinsamen Template-Verzeichnis.

3.4.2 Beispiele:

```
# Standard-Build aus aktuellem Verzeichnis
markpublish build

# HTML-Version in ein bestimmtes Zielverzeichnis ausgeben
markpublish build markpublish.yaml -t html -o dist/

# Benutzerdefiniertes Template-Verzeichnis nutzen
markpublish build --templates-dir /shared/company-templates
```

3.5 markpublish templates

Zeigt eine tabellarische Übersicht aller Templates auf dem System und deren Prioritätsstatus:

```
markpublish templates
markpublish templates --target pdf
```

3.6 markpublish export-template

Kopiert das eingebaute Standard-Template in Ihr lokales Arbeitsverzeichnis:

```
markpublish export-template default templates
```

3.7 markpublish labels

Zeigt, welcher statische Text am Ende gilt und aus welcher Ebene der Label-Kaskade er stammt. Nützlich, sobald ein Theme oder ein Dokument eigene Texte mitbringt und nicht mehr offensichtlich ist, welche Ebene gewinnt.

```
markpublish labels                # alle Schlüssel, Zielformat PDF
markpublish labels --target html  # Kaskade für die HTML-Ausgabe
markpublish labels --overridden   # nur überschriebene Texte
markpublish labels pfad/zu/markpublish.yaml
```

3.7.1 Argumente & Optionen

PARAMETER	TYP	STANDARD	BESCHREIBUNG
config_file	Pfad	markpublish.yaml	Pfad zur Konfigurationsdatei
--target, -t	String	pdf	Zielformat, dessen Kaskade gezeigt wird (pdf oder html)
--templates-dir	Pfad	-	Alternativer Template-Ordner
--overridden	Flag	aus	Nur Texte anzeigen, die das Theme ändert

Die Spalte *Source* nennt die Ebene: `i18n.yaml (de)` für den Programmstandard oder den Pfad einer Theme- bzw. Zielformat- `i18n.yaml`. Unter der Tabelle steht, in welchen Dateien nach Overrides gesucht wurde und welche existieren.

💡 Tipp

`--overridden` beantwortet die häufigste Frage direkt: *Was weicht in diesem Projekt überhaupt vom Standard ab?*

KAPITEL 4

Konfigurations-Leitfaden

Deklarative Definition von Dokumentenmetadaten, Kapiteln und Hierarchien.

AUF EINEN BLICK

4.1 Das document-Objekt	14
-------------------------------	----

4 Konfigurations-Leitfaden

Die Datei `markpublish.yaml` ist das zentrale Steuerungsdokument jeder Publikation. Sie gliedert sich in Metadaten, Template-Einstellungen und Kapiteldefinitionen.

4.1 Das `document`-Objekt

Unter `document:` werden alle globalen Metadaten und Schalter hinterlegt:

```
document:
  title: "markpublish Benutzerhandbuch"
  subtitle: "Moderne PDF- und HTML-Dokumentenerstellung"
  summary: "Kurze Zusammenfassung für Deckblatt und Metadaten."
  author: "Frank Winter"
  organisation: "WASDCAT Games"
  date: "auto"                # "auto" (Tagesdatum) oder festes Datum "2026-08-31"
  version: "1.0.0"           # optional - ohne Angabe entfällt das Feld
  language: "de"             # Sprachcode für Silbentrennung & Formatierung

# Globale Schalter
cover: true                  # Deckblatt an/aus
toc: true                    # Globales Inhaltsverzeichnis
autonum_type: "decimal"      # "decimal", "roman", "Legal", "none"
header: true                 # Laufende Kopfzeile
footer: true                 # Laufende Fußzeile
```

4.1.1 Automatische Datumsauflösung

Wird `date: "auto"` oder `date: "today"` angegeben, generiert `markpublish` automatisch das aktuelle Datum im passenden Sprachformat: - `language: "de"` `$\rightarrow$ 31.08.2026` - `language: "en"` `$\rightarrow$ 2026-08-31`

4.1.2 Zusätzliche Metadatenfelder

Beliebige zusätzliche Schlüssel (z. B. `status: "Entwurf"`, `department: "IT-Architektur"`) können frei definiert werden und stehen in allen Jinja2-Templates über `document.<feldname>` zur Verfügung.

4.2 Kapitel, Unterkapitel und Parts

markpublish unterstützt eine flexible und intuitive Strukturierung von Dokumenten.

4.2.1 Kapitel-Definition

Ein einfaches Kapitel wird mit `file:`, optionalem `title:` und `summary:` angegeben:

```
chapters:
- file: "chapters/01_intro.md"
  title: "Einleitung"
  summary: "Zielsetzung des Leitfadens."
  break_before: "divider"      # Eigene Trennseite vor dem Kapitel
  toc: false                   # Kein kapitelweises Mini-TOC
```

4.2.2 Hierarchische Unterkapitel

Um ein Unterkapitel zu definieren, rücken Sie einfach weitere Kapitel unter `chapters:` ein:

```
chapters:
- file: "chapters/02_architecture.md"
  title: "Systemarchitektur"
  break_before: "divider"
  toc: 2                      # Mini-TOC bis Überschriftstiefe 2
  chapters:
  - file: "chapters/02_1_backend.md"
    title: "Backend Services"
  - file: "chapters/02_2_frontend.md"
    title: "Frontend Client"
    break_before: "none"      # Läuft ohne Umbruch weiter
```

markpublish nummeriert die Unterkapitel automatisch konsistent als 2.1 und 2.2.

Ohne Angabe beginnt jedes Kapitel oben auf einer neuen Seite — auch ein Unterkapitel. Wie weit ein Kapitel abgesetzt wird, steuert durchgehend `break_before:` (`page`, `divider`, `none`); die Werte sind in Anhang A beschrieben.

4.2.3 Übergeordnete Abschnitte (Parts / Blöcke)

Für Hauptabschnitte oder Anhangsblöcke, die mehrere Kapitel umfassen, verwenden Sie das Schlüsselwort `part:`:

```
chapters:
- part: "Anhänge"
  summary: "Ergänzende Tabellen und Referenzen."
  break_before: "divider"      # Große Trennseite für den gesamten Anhang
  autonum: "none"              # Keine vorangestellte Ziffer
  toc_depth: 1                 # Anhänge nur mit Titel ins Inhaltsverzeichnis
  chapters:
    - file: "chapters/appendix_a.md"
      title: "Anhang A: Referenz"
    - file: "chapters/appendix_b.md"
      title: "Anhang B: Glossar"
```

Der Part-Titel (z. B. *“Anhänge”*) wird automatisch in die laufende Kopfzeile der Einzelseiten übernommen.

KAPITEL 5

Das Template- und Design-System

3-stufige Auflösungshierarchie, CSS Paged Media und 2-zeilige Kopf-/Fußzeilen.

AUF EINEN BLICK

5.1 Theme-basierte Verzeichnisstruktur	18
5.2 Die 3-stufige Auflösungs-Hierarchie	18
5.3 Statische Texte und Sprache	19
5.4 Kopf- und Fußzeilen	24

5 Das Template- und Design-System

Das Template-System von `markpublish` ermöglicht vollständige visuelle Anpassbarkeit bei gleichzeitiger Wartbarkeit.

5.1 Theme-basierte Verzeichnisstruktur

Templates sind nach Theme gruppiert, darunter liegt das Ausgabeformat. So gehören alle Dateien eines Designs zusammen und ein Theme lässt sich als Ganzes kopieren, weitergeben oder versionieren:

```
templates/  
└─ default/                                # Theme-Name  
    └─ pdf/  
        └─ layout.html                    # HTML-Grundgerüst  
        └─ styles.css                    # CSS Paged Media & Kopf-/Fußzeilen  
        └─ fonts/                        # Mitgelieferte Schrift (Open Sans, variabel) + OFL.txt  
        └─ cover.html                    # Deckblatt-Template  
        └─ part_divider.html              # Trennseite für übergeordnete Parts  
        └─ chapter_divider.html          # Trennseite für Kapitel mit Mini-TOC  
        └─ toc.html                      # Globales Inhaltsverzeichnis  
    └─ html/  
        └─ layout.html                    # Standalone Web-Layout  
        └─ styles.css                    # Screen/Responsive Stylesheet  
    └─ ...
```

Hinweis

Bis einschließlich Version 0.1 lag die Struktur umgekehrt (`templates/pdf/default`). Templates im alten Layout werden weiterhin gefunden, `markpublish templates` markiert sie in der Spalte *Layout* und gibt einen Hinweis aus. Verschieben Sie sie bei Gelegenheit — die alte Auflösung entfällt in einer künftigen Version.

5.2 Die 3-stufige Auflösungs-Hierarchie

Beim Suchen nach einem Template (z. B. `default/pdf`) gilt folgende strikte Priorität:

1. User-Verzeichnis (`~/.markpublish/templates/default/pdf/`)
| (höchste Priorität)
▼
2. Gemeinsamer / Projekt-Ordner (`<templates_dir>/default/pdf/`)
|
▼
3. Paket Built-in (im `markpublish` Python-Paket)

5.2.1 Konfiguration des gemeinsamen Template-Ordners

Sie können den gemeinsamen Template-Pfad auf 4 Wegen festlegen: 1. **CLI-Parameter:** `markpublish build --templates-dir /pfad/zu/templates` 2. **In `markpublish.yaml`:** `templates_dir: "./custom-templates"` 3. **Umgebungsvariable:** `MARKPUBLISH_TEMPLATES_DIR=/pfad/zu/templates` 4. **Automatischer Fallback:** `./templates` im Projektordner.

5.3 Statische Texte und Sprache

Die festen Beschriftungen — Überschrift des Inhaltsverzeichnisses, Kapitel-Marken, Cover-Labels, Seitenzahl-Fußzeile, Callout-Titel — stehen nicht im Template, sondern in `i18n.yaml`-Dateien. Welche Sprache gilt, bestimmt `document.language`:

```
document:
  title: "User Guide"
  language: "en"      # steuert Beschriftungen, Callout-Titel und Datumsformat
```

Mitgeliefert sind `de` und `en`. Regionale Formen werden zugeordnet (`de-AT` → `de`), eine unbekannte Sprache fällt auf Englisch zurück.

Hinweis

`i18n` bezeichnet die Quellen — die Dateien und den YAML-Eintrag, jeweils über alle Sprachen. `labels` ist das daraus aufgelöste Ergebnis für *ein* Dokument in *einer* Sprache: das, was im Template unter `{{ labels.chapter }}` ankommt.

5.3.1 Die 3-stufige i18n-Kaskade

Alle drei Ebenen sind **identisch aufgebaut**: Sprachcode auf oberster Ebene, darunter die Texte. Jede tiefere Ebene überschreibt die höher liegende — und zwar **nur die Schlüssel, die sie tatsächlich setzt**. Alles andere bleibt, wie es weiter oben steht:

1. Programm	<code>markpublish/i18n.yaml</code>
	vollständig, <code>de</code> + <code>en</code>
▼	
2. Theme	<code><templates>/<theme>/i18n.yaml</code>
	gilt für alle Zielformate des Themes
▼	
3. Zielformat	<code><templates>/<theme>/<target>/i18n.yaml</code>
	nur für PDF bzw. nur für HTML

Ebene 1 ist die einzige, die vollständig sein muss. Sie legt zusätzlich Englisch unter die Dokumentsprache, damit jeder Programmtext garantiert auflöst. Die Ebenen 2 und 3 sind reine Overrides.

Statische Texte werden **ausschließlich im Theme** definiert. Eine Dokumentebene gibt es nicht:

`document.i18n` in der `markpublish.yaml` wird abgelehnt, mit Hinweis auf die Theme-Datei. Wer die Texte eines Dokuments ändern will, gibt ihm sein eigenes Theme — `markpublish export-template` legt es an.

! Wichtig

Die Ebenen 2 und 3 stammen immer aus **genau einem** Theme: welches gilt, entscheidet vorher die Template-Auflösung (User > Projekt > Paket). Ein Projekt-Theme erbt **nicht** die Texte des gleichnamigen Paket-Themes — gemischt wird nur das eine gefundene Theme mit dem Programmstandard.

! Wichtig

Die Ebenen 2 und 3 greifen **nur für die gewählte Sprache**. Ein Theme, das einen `en:`-Block definiert, verändert eine deutsche Ausgabe nicht — sonst würden englische Theme-Texte in fremdsprachige Dokumente durchschlagen.

5.3.2 Das gemeinsame Format

```
# templates/mytheme/i18n.yaml
de:
  part: "Abschnitt"
  chapter_toc_title: "Auf dieser Seite"
en:
  part: "Section"
  chapter_toc_title: "On this page"
```

Der Sonderschlüssel `"*"` gilt für **jede** Sprache und wird vor dem sprachspezifischen Block angewendet — für Begriffe, die unabhängig von der Dokumentsprache gleich heißen:

```
"*":
  version: "Rev."      # in jeder Sprache "Rev."
de:
  part: "Abschnitt"
```

Wer nur eine Sprache pflegt, darf die Sprachebene auch weglassen; die flache Form ist gleichbedeutend mit `"*"`:

```
part: "Abschnitt"      # entspricht: "*" : \n part: "Abschnitt"
```

Regionale Blöcke schlagen den Basis-Block: bei `language: "de-AT"` gewinnt `de-at:` über `de:`. Fehlt eine `i18n.yaml`, ist das kein Fehler — ein Theme ohne eigene Texte ist der Normalfall. Ist sie vorhanden, aber fehlerhaft, bricht der Build mit Angabe der Datei ab, statt still die Standardtexte zu verwenden.

5.3.3 Beispiel: PDF und HTML unterschiedlich beschriften

```
templates/mytheme/  
├─ i18n.yaml           de: chapter: "Kapitel"  
├─ pdf/  
│   └─ i18n.yaml       de: chapter: "Kap."      ← nur im PDF  
└─ html/  
    └─ i18n.yaml       (leer → erbt "Kapitel")
```

Das mitgelieferte Theme `default` bringt alle drei Dateien als **auskommentiertes Muster** mit: `templates/default/i18n.yaml`, `default/pdf/i18n.yaml` und `default/html/i18n.yaml`. Sie sind bewusst wirkungslos — das Standard-Theme soll exakt wie der Programmstandard aussprechen. Kommentieren Sie aus, was Sie ändern möchten. `markpublish export-template` kopiert diese Dateien mit — auch die `i18n.yaml` der Theme-Ebene, die neben den Zielformat-Ordnern liegt.

5.3.4 Freie Labels: eigene Texte des Templates

Ein Theme darf **eigene Schlüssel** definieren, die das Programm nicht kennt — für die statischen Texte des Templates selbst. Der Aufbau ist derselbe, der Zugriff ebenso:

```
# templates/mytheme/i18n.yaml  
de:  
  imprint_title: "Impressum"  
  disclaimer: "Alle Angaben ohne Gewähr."  
en:  
  imprint_title: "Imprint"  
  disclaimer: "All information without guarantee."
```

```
<!-- templates/mytheme/html/Layout.html -->  
<footer>{{ labels.imprint_title }}</footer>
```

⚠️ Warnung

Für freie Labels gibt es **keinen Programmstandard**, der einspringen könnte. Deshalb gilt hier eine strikte Regel: Ein Label, das ein Template notiert, **muss** in der Kaskade auflösen. Tut es das nicht, bricht der Build ab und nennt Schlüssel, Fundstelle mit Zeilennummer, Dokumentsprache und die durchsuchten Dateien:

```
Label 'imprint_title' ist im Template notiert, aber in keiner i18n-Ebene definiert.  
Dokumentsprache: de  
Fundstelle:  
  templates/mytheme/html/layout.html:108  
Gesucht in:  
  templates/mytheme/html/i18n.yaml (vorhanden)  
  templates/mytheme/i18n.yaml      (vorhanden)  
  markpublish/i18n.yaml            (vorhanden)
```

Pflegen Sie also jede Sprache, die Sie ausliefern, oder legen Sie den Schlüssel unter `"*"` ab. Ein leerer Text im fertigen PDF fällt niemandem auf — ein Abbruch schon.

Geprüft werden die Template-Quellen, nicht der Renderlauf: auch ein Label in einem Zweig, den genau dieses Dokument nicht durchläuft, wird gemeldet. `styles.css` zählt mit, da es durch dieselbe Jinja-Umgebung läuft.

5.3.5 Die aufgelöste Tabelle ansehen

Bei drei Ebenen ist nicht immer offensichtlich, woher ein Text kommt. `markpublish labels` zeigt das Ergebnis samt Herkunft:

```
markpublish labels          # alle Schlüssel, Ziel PDF  
markpublish labels --target html # Kaskade für die HTML-Ausgabe  
markpublish labels --overrides # nur das, was vom Theme kommt
```

5.3.6 Verfügbare Schlüssel

SCHLÜSSEL	DE	EN
<code>toc_title</code>	Inhaltsverzeichnis	Table of Contents
<code>toc_sidebar</code>	Inhalt	Contents
<code>chapter_toc_title</code>	Inhalt dieses Kapitels	In this chapter
<code>chapter</code>	Kapitel	Chapter
<code>part</code>	Teil	Part
<code>author</code>	Autor	Author
<code>status</code>	Status	Status
<code>version</code>	Version	Version
<code>date</code>	Datum	Date
<code>copyright</code>	Copyright	Copyright
<code>page</code>	Seite	Page
<code>page_of</code>	von	of
<code>alert_note</code>	Hinweis	Note
<code>alert_tip</code>	Tipp	Tip
<code>alert_important</code>	Wichtig	Important
<code>alert_warning</code>	Warnung	Warning
<code>alert_caution</code>	Achtung	Caution

Die `alert_*`-Titel entstehen bereits beim Markdown-Parsen, nicht erst im Template — die Kaskade wird dorthin durchgereicht, ein `alert_note` im Theme wirkt also auch im Callout.

Eine neue Sprache legen Sie an, indem Sie in `markpublish/i18n.yaml` einen vollständigen Block ergänzen — oder, ohne das Paket anzufassen, auf Ebene 2 oder 3 alle Schlüssel unter dem gewünschten Sprachcode setzen.

💡 Tipp

In eigenen Templates greifen Sie mit `{{ labels.chapter }}` auf das Ergebnis zu — auch in `styles.css`, das durch dieselbe Jinja-Umgebung läuft. So ist die Fußzeile `"{{ labels.page }}" counter(page) "` `{{ labels.page_of }}" counter(pages)` gebaut.

5.4 Kopf- und Fußzeilen

Kopf- und Fußzeile sind **Running Elements**: ein Block im `layout.html` bekommt

`position: running(name)` und wird damit aus dem Textfluss genommen; die `@page`-Regel setzt ihn über `content: element(name)` in die Margin-Box ein.

Der Unterschied zu einer `content:`-Zeichenkette ist wesentlich: In der Margin-Box steht dadurch **echtes Markup**. Damit sind beliebig viele Zeilen möglich, jede mit eigener Auszeichnung — eine Zeichenkette kennt nur eine Formatierung für alles.

```
<!-- layout.html -->
<div class="page-header-runner">
  <div class="hf-left">
    <div class="hf-doc-title">{{ document.title }}</div>
    <div class="hf-doc-subtitle">{{ document.subtitle }}</div>
  </div>
  <div class="hf-right"><span class="hf-section"></span></div>
</div>
```



```
/* styles.css */
.page-header-runner {
  position: running(pageheader);
  display: flex;
  justify-content: space-between;
  align-items: flex-start; /* rechte Spalte bleibt oben */
}
.hf-doc-title { font-weight: 700; }
.hf-section::before { content: string(current-section); }

@page {
  @top-left {
    content: element(pageheader);
    width: 100%;
    vertical-align: top;
    margin-top: 12mm; /* Abstand zur Papierkante */
    padding-bottom: 0;
    border-bottom: 0.5pt solid #cbd5e1;
    margin-bottom: 12mm; /* Abstand zum Inhalt */
  }
}
```

5.4.1 Geometrie

Von der Papierkante nach innen:

Kante — margin — Zeilen (top-aligned) — Linie — margin — Inhalt
12 mm 12 mm

Beide Abstände sind bewusst gleich groß: eine Kopfzeile, die dicht über dem Text sitzt, liest sich als Teil des Satzspiegels statt als Seitenfurnitur.

Beide Spalten sitzen in einem Flex-Container mit `align-items: flex-start`. Die rechte Spalte beginnt deshalb auf der Höhe der **ersten** linken Zeile, auch wenn links drei Zeilen stehen und rechts nur eine.

Wichtig

Der Abstand zum Inhalt kommt aus `margin-bottom`, nicht aus `padding-bottom`. In CSS liegt der Rahmen **außerhalb** des Paddings — ein `padding-bottom` würde die Trennlinie vom Text wegschieben und an den Inhalt drücken. Gewollt ist das Gegenteil: Linie direkt am Text, Abstand danach.

⚠️ Warnung

Eine Margin-Box **schiebt den Inhalt nicht**. Ihre Höhe ist durch den Seitenrand gedeckelt; zusätzliche Zeilen laufen aus der Seite heraus, statt den Satzspiegel zu verkleinern. Der Seitenrand wird deshalb in `styles.css` aus der Zeilenzahl gerechnet:

```
margin-top = Rand zur Kante + Zeilen × Zeilenhöhe + Linienstärke + Abstand zum Inhalt
```

Wer in `layout.html` eine Zeile ergänzt, zieht `hf_header_lines` bzw. `hf_footer_lines` in `styles.css` mit.

5.4.2 Standardbelegung des Themes

Kopfzeile	Dokumenttitel (fett) Untertitel	Kapiteltitel
Fußzeile	Copyright	Version 1.0.0 01.09.2026 Seite X von Y

Untertitel und Version sind optional. Fehlt der Untertitel, hat die Kopfzeile nur eine Zeile und der Satzspiegel rückt entsprechend nach oben. Fehlt die Version, entfällt sie samt Trenner — in der Fußzeile steht dann nur das Datum, und auf dem Deckblatt fehlt das Feld ganz.

Der Kapiteltitel kommt aus `string(current-section)`, das die Kapitel- `<article>` per `string-set` setzen; `counter(page)` funktioniert innerhalb des Running Elements ebenso wie in einer Margin-Box. Die Schalter `document.header` und `document.footer` blenden den jeweiligen Block ab; auf Deck- und Trennseiten ist er ohnehin abgeschaltet.

KAPITEL 6

Markdown-Features & Syntax

Tabellen, Admonitions/Callouts, Pygments Syntax-Highlighting und Tasklisten.

6 Markdown-Features & Syntax

markpublish beinhaltet eine leistungsfähige Markdown-Engine mit voller Unterstützung moderner Dokumentations-Elemente.

6.1 Code-Blöcke & Syntax Highlighting

Quellcode wird durch Pygments hervorgehoben:

```
from markpublish.config.loader import load_config
from markpublish.markdown.engine import MarkdownPipeline

# Manifest Laden
config = load_config("markpublish.yaml")
print(f"Building: {config.document.title}")
```

6.2 Tabellen

Tabellen werden mit sauberem CSS-Zebramuster und Seitenumbruchschutz gerendert:

PARAMETER	TYP	STANDARD	BESCHREIBUNG
title	String	<i>Pflicht</i>	Haupttitel des Dokuments
author	String	None	Name des Autors
status	String	None	Dokumentstatus (z.B. "Freigegeben", "Draft")
copyright	String	None	Copyright-Vermerk (z.B. "© 2026 Frank Winter")
version	String	null	Versionskennung. Ohne Angabe entfällt das Feld auf dem Deckblatt; ist sie gesetzt, steht sie in der Fußzeile links neben dem Datum

6.3 GitHub-Style Callout-Boxen (Alerts)

markpublish unterstützt die gängige **GitHub-Alert-Syntax** mit fünf semantischen Typen. Die Icons stammen dabei direkt aus dem Template:

Hinweis

Dies ist eine informative Notiz mit blauem Akzent und passendem Info-Icon.

Tipp

Nutzen Sie `markpublish build --target all`, um PDF und HTML parallel in einem Durchgang zu erstellen.

Wichtig

GitHub-Callouts werden automatisch anhand der Dokumentensprache lokalisiert (z. B. *Hinweis*, *Tipp*, *Wichtig*, *Warnung*, *Achtung*).

Warnung

Achten Sie bei relativen Bildpfaden darauf, dass diese relativ zur jeweiligen Markdown-Datei liegen.

Achtung

Fehlerhafte YAML-Einrückungen führen zu Abbruchfehlern beim Parsen des Manifests.

Eigener Titel

Sie können nach dem Alert-Typ auch einen individuellen Titel angeben, der die Standardbeschriftung überschreibt.

Alternativ wird auch weiterhin die klassische `!!! note`-Syntax unterstützt.

6.4 Tasklisten

- ☒ Projekt initialisieren
- ☒ Kapitel schreiben
- ☐ Dokument veröffentlichen

ABSCHNITT

Anhänge

Vollständige Spezifikations-Tabellen, Troubleshooting und Best Practices.

6.5 Anhang A: YAML-Schema-Referenz

Vollständige Übersicht aller Konfigurationsoptionen in `markpublish.yaml`.

6.5.1 Dokumenten-Eigenschaften (`document`)

SCHLÜSSEL	TYP	STANDARD	BESCHREIBUNG
<code>title</code>	String	<i>Pflicht</i>	Titel der Publikation
<code>subtitle</code>	String	<code>null</code>	Untertitel
<code>summary</code>	String	<code>null</code>	Zusammenfassung für Deckblatt
<code>author</code>	String	<code>null</code>	Name des Autors
<code>status</code>	String	<code>null</code>	Dokumentstatus (z. B. "Entwurf", "Freigegeben")
<code>copyright</code>	String	<code>null</code>	Copyright-Angabe (z. B. "© 2026 Frank Winter")
<code>date</code>	String	<code>"auto"</code>	Datum oder <code>"auto"</code> für Tagesdatum
<code>version</code>	String	<code>null</code>	Versionskennung. Ohne Angabe entfällt das Feld auf dem Deckblatt; ist sie gesetzt, steht sie in der Fußzeile links neben dem Datum
<code>language</code>	String	<code>"de"</code>	ISO-Sprachcode (<code>de</code> , <code>en</code> , ...). Steuert Template-Beschriftungen, Callout-Titel und Datumsformat
<code>cover</code>	Bool	<code>true</code>	Deckblatt aktivieren/deaktivieren
<code>toc</code>	Bool	<code>true</code>	Globales Inhaltsverzeichnis aktivieren
<code>autonom_type</code>	String	<code>"decimal"</code>	<code>"decimal"</code> , <code>"roman"</code> , <code>"legal"</code> , <code>"none"</code>
<code>header</code>	Bool	<code>true</code>	Laufende Kopfzeile aktivieren (Aufbau im Theme)
<code>footer</code>	Bool	<code>true</code>	Laufende Fußzeile aktivieren (Aufbau im Theme)

6.5.2 Kapitel- und Part-Eigenschaften (chapters)

SCHLÜSSEL	TYP	STANDARD	BESCHREIBUNG
file	String	null	Pfad zur Markdown-Datei
title	String	null	Überschreibt den Titel der Datei
summary	String	null	Zusammenfassung für Trennseite
part	String	null	Deklariert einen übergeordneten Part
break_before	String	page	Wie das Kapitel abgesetzt wird: <code>page</code> , <code>divider</code> oder <code>none</code> . Nur PDF — das mitgelieferte HTML-Theme kennt weder Seiten noch Trennseiten
toc	Bool/ Int	false	Lokales Kapitel-TOC (z. B. <code>2</code> für max. Tiefe). Steht auf der Trennseite, also ebenfalls nur PDF
toc_depth	Int	null	Begrenzt, wie tief dieser Zweig ins globale Inhaltsverzeichnis einzieht
autonum	String	null	Lokaler Override des Nummerierungsstils
chapters	Liste	[]	Verschachtelte Unterkapitel

6.5.2.1 break_before — wie ein Kapitel abgesetzt wird

Ein Schlüssel mit drei Werten, keine zwei unabhängigen Schalter:

WERT	WIRKUNG
page	Standard. Das Kapitel beginnt oben auf einer neuen Seite
divider	Dem Kapitel geht eine eigene Trennseite voraus
none	Das Kapitel läuft im Fließtext weiter

Dass es *ein* Schlüssel ist, hat einen Grund: die drei Werte sind eine Achse, nicht drei Fragen. Als zwei Booleans ließe sich „Trennseite, aber kein Seitenumbruch“ hinschreiben — ein Zustand, den es nicht gibt, weil eine Trennseite immer umbricht. Eine Angabe, die man notieren kann und die nichts bewirkt, ist eine Fehlerquelle ohne Gegenwert.

`page` ist die Erwartung an ein gesetztes Dokument. Wer kurze Abschnitte durchlaufen lassen will — Merkbblätter, Referenzkarten, eng gesetzte Anhänge — setzt am einzelnen Kapitel `break_before: "none"`; die Vorgabe lässt sich auch im Frontmatter der Markdown-Datei überschreiben.

Bei `divider` fallen der Umbruch des Kapitels und der der Trennseite auf dieselbe Stelle und verschmelzen zu einem — es entsteht kein Leerblatt. Vor dem ersten Kapitel greift die Regel nicht.

Ein unbekannter Wert bricht den Build ab, statt still auf `page` zurückzufallen: aus einem vertippten `divder` würde sonst klaglos ein normaler Seitenumbruch, und die fehlende Trennseite fände man erst beim Durchblättern des fertigen PDFs.

6.5.2.2 `toc_depth` — Anhänge im Inhaltsverzeichnis flach halten

Gezählt wird innerhalb des Kapitels, genau wie bei `toc`: Tiefe 1 ist die Kapitelüberschrift selbst, Tiefe 2 die Ebene darunter. `toc_depth: 1` nimmt das Kapitel also ins globale Inhaltsverzeichnis auf, seine Zwischenüberschriften aber nicht mehr.

Der Wert wird nach unten vererbt: an einem Part gesetzt, gilt er für alle Kapitel darunter, und ein Kapitel gibt ihn an seine Unterkapitel weiter. Für den häufigsten Fall — ein Anhang, dessen innere Gliederung das Verzeichnis nur aufbläht — genügt damit eine Zeile:

```
- part: "Anhänge"
  toc_depth: 1
  chapters:
    - file: "chapters/07_appendix_yaml_spec.md"
      title: "Anhang A: YAML-Schema-Referenz"
    - file: "chapters/08_appendix_troubleshooting.md"
      title: "Anhang B: Troubleshooting"
```

Der Fließtext bleibt unberührt: die Zwischenüberschriften stehen weiterhin im Kapitel, samt Nummerierung und Sprungzielen. Gekürzt wird ausschließlich das Verzeichnis. Ein Unterkapitel behält dabei seinen eigenen Eintrag — vererbt wird die Tiefe relativ zu jedem Kapitel, nicht über die zusammengelegte Liste.

6.6 Anhang B: Fehlerbehebung & FAQ

Häufige Fragen und Lösungen bei der Dokumentenerstellung.

6.6.1 WeasyPrint & GTK-Laufzeitumgebung auf Windows

6.6.1.1 Problem: `cannot load library 'libobject-2.0-0'`

WeasyPrint benötigt auf Windows-Systemen die nativen C-Bibliotheken von Pango und GTK.

Lösung: `markpublish` sucht automatisch nach installierten GTK-Umgebungen (wie MSYS2, GTK3-
Runtime, darktable oder Inkscape). Sollten diese fehlen, installieren Sie das offizielle GTK3-
Runtime-Paket: - Download: [GTK-for-Windows-Runtime-Environment-Installer](#) - Alternativ via MSYS2: `pacman -S
mingw-w64-x86_64-pango`

6.6.2 Bilder werden im PDF nicht angezeigt

6.6.2.1 Problem: Leere Bildrahmen oder fehlende Grafiken

WeasyPrint benötigt gültige relative Pfade oder absolute File-URIs.

Lösung: Geben Sie relative Bildpfade immer relativ zur jeweiligen Markdown-Datei an:

```
![Architekturdiagramm](images/architecture.png)
```

`markpublish` löst relative Pfade automatisch zu absoluten Datei-URIs auf.

6.6.3 Inhaltsverzeichnis zeigt falsche Seitenzahlen

Das PDF-Rendering erfolgt in mehreren Layout-Durchläufen (Zweipass-Rendering von WeasyPrint).
Stellen Sie sicher, dass alle internen Überschriften-IDs eindeutig sind – `markpublish` übernimmt diese
Eindeutigkeit automatisch über seinen internen Slug-Generator.